Árbol de Steiner consciente de tokens para consultas de movilidad urbana en LLMs

David Rangel Monsiváis, Damián Alejandro Gómez Mena, Gael Alexandor Zaragoza Torres
Equipo 7 · TC2038 Análisis y Diseño de Algoritmos Avanzados · Tecnológico de Monterrey
Midterm 1 · 21 de octubre de 2026

***Resumen*—Un LLM que responde consultas de movilidad no puede recibir el grafo vial de una zona metropolitana: el grafo de solo el centro de Monterrey serializado ocupa 254,893 tokens. Proponemos una herramienta MCP que, dada la consulta ya interpretada por el LLM (origen, destino y puntos de interés), devuelve un subgrafo podado mediante un Árbol de Steiner con pesos conscientes de tokens, una ruta ordenada exacta sobre ese subgrafo (Held–Karp) con reparación dirigida, y una serialización con contracción de cadenas. Presentamos la formalización, la correctitud, la complejidad y una cota de aproximación para la relajación simétrica, junto con un experimento preliminar sobre 150 consultas fijas en tres zonas de OpenStreetMap: el método reduce los tokens del prompt entre 9× y 119× frente al grafo completo (la razón crece con el grafo), con 100 % de rutas válidas, costo medio a 2.5–4.9 % del óptimo y salida determinista bit a bit. También reportamos un resultado negativo: el peso por arista consciente de tokens no reduce el texto serializado, porque ignora la contracción; proponemos la corrección.**

I. Panorama del proyecto

Problema real. Las consultas de navegación en lenguaje natural traen varios puntos de interés (POI) y preferencias («del Tec a la Macroplaza pasando por una farmacia y un banco, sin peajes»). Enviar el mapa completo al LLM excede la ventana de contexto, encarece cada consulta de forma lineal en tokens y degrada la calidad: los modelos usan mal la información enterrada en contextos largos [1] y pueden inventar calles. **Arquitectura.** El LLM actúa como orquestador vía MCP [2]: (1) extrae entidades y restricciones; (2) llama a la herramienta `plan_route`; (3) nuestro algoritmo filtra, poda y ordena; (4) devuelve un texto compacto de a lo más B tokens; (5) el LLM verbaliza las instrucciones. El *LLM decide* qué restricciones semánticas aplicar y cómo redactar; el *algoritmo decide* qué calles entran al contexto y en qué orden se visitan los POI. **Requisitos.** Recurso optimizado: tokens de entrada (costo y latencia de *prefill*). Calidad: ruta 100 % válida sobre el grafo, cobertura total de POI, costo a no más de 10 % del óptimo ($\rho \leq 1.1$). Datos: OpenStreetMap vía OSMnx [3], usado solo para obtener el grafo; Dijkstra, Steiner, Held–Karp, Kruskal y la serialización son implementación propia.

Parte I — Análisis algorítmico

II. Q1. Definición, motivación y supuestos

Entradas. Un grafo dirigido $G = (V, E)$, $n = |V|$, $m = |E|$, fuertemente conexo, con tiempo de recorrido $t : E \to \mathbb{R}_{>0}$ y nombre de calle por arista. Una consulta $q = (s, d, P, C)$ con origen s, destino d, POI $P = \{p_1, \ldots, p_k\} \subseteq V$ y restricciones C (aristas prohibidas). Terminales $S = \{s, d\} \cup P$, $\ell = |S|$. Un presupuesto de contexto B y un serializador σ con conteo de tokens $\mathrm{tok}(\cdot)$.

Salidas. Un subgrafo $H \subseteq G_C$ (G sin las aristas de C) y un recorrido $R = (s, \ldots, d)$ en H que visita todo P, entregados como el texto $\sigma(H, R)$.

Objetivos (en conflicto).

$$f_1 = \mathrm{tok}(\sigma(H, R)) \qquad \text{(dominio computacional)} \quad (1)$$

$$f_2 = c(R) = \textstyle\sum_{e \in R} t(e) \qquad \text{(dominio físico)} \quad (2)$$

sujeto a: R válido en G (cada par consecutivo es arista), $P \subseteq R$, H conecta S y $f_1 \leq B$. Se escalariza con $w_\lambda(e) = t(e) + \lambda\, \tau(e)$, donde $\tau(e)$ es el costo en tokens de serializar e; la hipótesis es que barrer λ traza el frente de Pareto entre f_1 y f_2 (refutada para el τ actual, ver M2).

Modelo computacional. RAM con aritmética real; el LLM es una caja negra cuyo costo es $p_{\text{in}} f_1 + p_{\text{out}}\, \text{tokens}_{\text{out}}$ y cuya latencia de *prefill* crece al menos linealmente con f_1.

Por qué Steiner. Conectar exactamente S con la red más barata es el problema de Árbol de Steiner en grafos, NP-difícil [4]. Un camino Hamiltoniano sobre V sería incorrecto (obliga a visitar todo); concatenar caminos mínimos fija un orden arbitrario y no comparte calles. El Árbol de Steiner es la estructura mínima que contiene un camino entre *cualquier* par de terminales, por lo que sirve para cualquier orden de visita sin otra llamada a la herramienta.

Supuestos. (A1) El LLM resuelve cada POI a un nodo (*entity linking*); los POI por categoría quedan como trabajo futuro (Q5). (A2) Tiempos estáticos derivados de límites de velocidad de OSM. (A3) El árbol se calcula sobre la simetrización $t^{\leftrightarrow}(\{u, v\}) = \min(t(u, v), t(v, u))$, porque el Steiner dirigido solo admite aproximaciones $O(\ell^{\varepsilon})$ [5]; la dirección se recupera en la etapa de ruta. (A4) k pequeño ($k \leq 12$), realista para consultas humanas. (A5) `o200k_base` [6] como tokenizador de referencia.

III. Q2. Algoritmo central y correctitud

Componentes. Mehlhorn [7]: un Dijkstra multifuente [8] desde S particiona V en celdas de Voronoi; cada arista frontera (u, v) propone la arista (r_u, r_v) de costo $\delta(u) + w(u, v) + \delta(v)$; el MST (Kruskal) de ese grafo se expande a caminos, se recalcula el MST del subgrafo resultante y se podan hojas no terminales. HeldKarp [9]:

Algoritmo 1 TOKENSTEINERROUTE(G, s, d, P, B, Λ)

Entrada: grafo G, terminales, presupuesto B, rejilla creciente Λ

Salida: texto σ con $\mathrm{tok}(\sigma) \le B$ (o el mejor esfuerzo)

$\tau \leftarrow$ costo en tokens por arista ▷ precómputo, $O(m)$
for $\lambda \in \Lambda$ en orden creciente **do**
  $U_\lambda \leftarrow$ simetrización de G con w_λ
  $T \leftarrow$ MEHLHORN(U_λ, S) ▷ Voronoi + MST + poda
  $H \leftarrow \{(u,v) \in E : \{u,v\} \in T\}$ ▷ reorientar
  $(R, H) \leftarrow$ RUTAORDENADA(G, H, s, d, P)
  $\sigma \leftarrow$ SERIALIZAR(H, R) ▷ contracción de cadenas
  if $\mathrm{tok}(\sigma) \le B$ **then return** σ
return σ marcado como *fuera de presupuesto*

procedure RUTAORDENADA(G, H, s, d, P)
  for $a \in S$; $b \in S \setminus \{a\}$ **do**
    $D[a,b], \pi_{ab} \leftarrow$ Dijkstra dirigido en H desde a
    if b inalcanzable en H **then** ▷ calle de un sentido
      $\pi_{ab} \leftarrow$ Dijkstra en G; $H \leftarrow H \cup \pi_{ab}$ ▷ reparación
  orden $\leftarrow$ HELDKARP(D, s, d, P)
  return concatenación de π según el orden, H

$\mathrm{DP}[X, j] = \min_{i \in X \setminus \{j\}} \mathrm{DP}[X \setminus \{j\}, i] + D[i, j]$ sobre subconjuntos $X \subseteq P$.

Contribución algorítmica. No aplicamos un Steiner "de catálogo": (i) el *peso consciente de tokens* w_λ y el lazo sobre λ que ajusta la poda al presupuesto B (implementado; su proxy actual falla, ver M2 y Q5); (ii) la *contracción de cadenas*: un nodo no terminal de grado 2 cuyas dos aristas son de la misma calle y el mismo sentido se funde, y los ids OSM se renombran a ids locales; (iii) la *ruta ordenada con reparación dirigida*, necesaria porque un árbol no es una ruta: no fija el orden de visita y, al reorientar, puede perder caminos dirigidos.

Invariantes. (I1) Dijkstra: al extraer u del heap, $\delta(u)$ es la distancia mínima (pesos no negativos) y r_u es el terminal más cercano. (I2) Kruskal: el conjunto elegido es siempre un bosque contenido en algún MST. (I3) Durante RUTAORDENADA, $H \subseteq E$ y toda distancia en D corresponde a un camino dirigido real. (I4) La poda solo elimina hojas $\notin S$, por lo que T sigue conectando S.

Proposición 1 (Validez)**.** *Si G es fuertemente conexo,* TOKENSTEINERROUTE *termina y devuelve una ruta R válida en G que inicia en s, termina en d y visita todo P.*

Esbozo. Por (I3) cada π_{ab} es un camino en G; si no existe en H la reparación lo toma de G, que existe por conexidad fuerte. Hay a lo sumo $\ell(\ell-1)$ reparaciones. La concatenación según el orden de Held–Karp empieza en s, termina en d y pasa por cada p_i. □

Proposición 2 (Contracción sin pérdida)**.** *Las distancias dirigidas entre nodos no contraídos y la conectividad de H son idénticas antes y después de contraer.*

Esbozo. Un nodo contraído x tiene exactamente dos vecinos y, si la calle es de un sentido, una arista entra y otra sale; todo camino que toca x lo atraviesa, así que reemplazar la cadena por una arista con tiempo sumado preserva caminos y costos (homeomorfismo de grafos). Los terminales nunca se contraen. □

Optimalidad. Held–Karp es exacto sobre la métrica D_H; si $H = G$, la ruta es óptima ($\rho = 1$), y así calculamos la referencia.

IV. Q3. Complejidad, garantías y límites

Tiempo. Dijkstra con heap binario: $O((n+m)\log n)$. MEHLHORN: un Dijkstra multi-fuente, un barrido de aristas y dos Kruskal: $O(m \log n)$ (con heap de Fibonacci [10], $O(m + n\log n)$). RUTAORDENADA: ℓ Dijkstra sobre H, $O(\ell\,|H| \log |H|)$, más $r \le \ell(\ell-1)$ reparaciones de $O(m \log n)$. Held–Karp: $O(2^k k^2)$ tiempo y $O(2^k k)$ espacio. Serialización: $O(|H|)$ más el tokenizador, lineal en la longitud del texto. Total:

$$O\big(|\Lambda|\,\big[(1+r)\,m\log n + \ell|H|\log|H| + 2^k k^2\big]\big)\,,$$

dominado por $m \log n$ para $k \le 12$. **Espacio:** $O(n+m+2^k k)$.

Garantías. Para el árbol, $w_\lambda(T) \le 2(1-1/\ell)\,\mathrm{OPT}^{\mathrm{St}}_\lambda$ [7], [11]; el mejor factor conocido es $\ln 4 + \varepsilon \approx 1.39$ [12], a un costo práctico mucho mayor. Para la ruta en la relajación simétrica sin reparaciones ($\lambda = 0$):

Lema 1. *$c(R) \le 4(1-1/\ell)\,c(R^*)$, con R^* la ruta óptima.*

Esbozo. Las aristas de R^* forman un subgrafo conexo que contiene S, luego $c(R^*) \ge \mathrm{OPT}^{\mathrm{St}}$. Un recorrido en profundidad de T desde s que termina en d visita todos los terminales con costo $\le 2w(T)$; Held–Karp sobre D_T no es peor. Por tanto $c(R) \le 2w(T) \le 4(1-1/\ell)\mathrm{OPT}^{\mathrm{St}} \le 4(1-1/\ell)c(R^*)$. □

La cota es de peor caso; empíricamente ρ es mucho menor (Sec. VII). Con calles de un sentido la cota no se sostiene (las reparaciones rompen la estructura de árbol), y la exactitud de la ruta depende de H, no de G.

Límites. Held–Karp es exponencial en k: para $k > 15$ se requiere una heurística (vecino más cercano + 2-opt). La monotonía de f_1 en λ no está garantizada (Mehlhorn es aproximado y τ es un proxy); el lazo usa λ creciente y se detiene en el primero que cumple B. El término $r\,m\log n$ no es teórico: observamos $r \approx 14$–19 reparaciones por consulta, y domina el tiempo en grafos grandes. **Paralelismo (HPC):** los ℓ Dijkstra de RUTAORDENADA y los valores de Λ son independientes (paralelismo trivial, sin comunicación salvo reunir D, $O(\ell^2)$ números); no es el foco del proyecto.

V. Q4. Robustez, escalabilidad y calidad

Comparación. *B0*: grafo completo (sin optimizar). *B1*: Dijkstra concatenado en el orden en que el usuario nombró los POI. *B2*: recuperación local tipo RAG [13]: calles a ≤ 250 m de la poligonal entre terminales. *M*: propuesto. *REF*: solo la ruta óptima (Held–Karp sobre G), cota inferior de tokens y

respuesta de referencia. Todos usan el mismo serializador, de modo que las diferencias se deben solo a la selección de calles.

Carga creciente. Tres ejes: tamaño del grafo (n de $\sim$600 a $\sim$ 12,000 nodos), número de POI ($k \in \{1,2,3,4,6\}$) y presupuesto B (barrido de λ). B0 crece como $\Theta(m)$ tokens y deja de caber en una ventana de 128k tokens con pocas decenas de miles de aristas; M crece con el tamaño del árbol, $O(\ell \cdot \text{diámetro})$, casi independiente de n. B2 depende de la geometría y no garantiza conectividad: su ruta puede no existir.

Métricas de calidad (condiciones idénticas). *Correctitud*: validez de la ruta (cada par consecutivo es arista de G) y $\rho = c(R)/c(R^*)$. *Completitud*: fracción de POI cubiertos y de restricciones C respetadas. A nivel LLM: la respuesta se convierte a una secuencia de calles y se compara con R (coincidencia exacta y distancia de edición normalizada); se cuentan calles inexistentes en H (alucinaciones). *Consistencia*: $N = 10$ repeticiones por consulta con el mismo modelo fijado por versión, misma plantilla, `top_p`$= 1$, semilla fija y $T \in \{0, 0.7\}$; se reporta la fracción de pares de respuestas con la misma secuencia de calles y la invarianza ante 3 paráfrasis de la consulta.

VI. Q5. Limitaciones y siguientes pasos

L1. El proxy de tokens está mal alineado (evidencia en M2): $\tau(e)$ por arista ignora la contracción. *Mejora:* cobrar tokens por *cambio de calle*, con peso en los nodos donde cambia el nombre; esto da un Steiner con pesos en nodos, aproximable a $O(\ln \ell)$ [14], o un Steiner sobre el grafo de líneas. *Evaluación:* correlación de Spearman entre λ y tokens (esperada negativa) y frente de Pareto no dominado en las mismas consultas. **L2. El árbol restringe la ruta**: $\rho_{95} = 1.11$–1.24 cuando el árbol no contiene el camino óptimo, y un árbol no ofrece rutas alternativas. *Mejora:* $H' = T \cup R^*$ garantiza $\rho = 1$ por construcción, pues Held–Karp sobre G ya se calcula; costo esperado: a lo más los tokens de REF adicionales. *Evaluación:* Δf_1 contra $\Delta\rho$ en las 150 consultas. **L3. POI por categoría** («una farmacia») convierten el problema en *Group Steiner Tree*, con aproximación polilogarítmica [15]. *Mejora:* preselección de g candidatos por categoría por vecinos más cercanos y Held–Karp generalizado sobre grupos, $O(2^k g^2 k^2)$. *Evaluación:* ρ contra la enumeración exhaustiva en instancias pequeñas.

Parte II — Reporte de avance

VII. M1 — Algoritmo central (implementado)

Implementado (Python 3.13, $\sim$400 líneas propias): Dijkstra multi-fuente, DSU/Kruskal, Steiner de Mehlhorn, poda de hojas, Held–Karp, ruta ordenada con reparación, peso w_λ, lazo de presupuesto, serializador (lista y JSON) con contracción, baselines B0–B2 y REF. Validado con 6 pruebas: Dijkstra coincide con NetworkX en distancias; el árbol es árbol, contiene S y sus hojas son terminales; Held–Karp coincide con fuerza bruta; toda ruta de M es válida y $\rho \geq 1$.

Resultados preliminares (Tabla I, Fig. 1). (1) *Eficiencia*: M usa 9.4×, 33.9× y 119× menos tokens que B0; B0 en el centro (254,893) ya excede ventanas de 128k y 200k tokens, mientras M crece de 1,094 a 2,136 tokens con un grafo 19 veces mayor. M cuesta 2.3–3× los tokens de REF (solo la ruta) a cambio de entregar la red que conecta todos los terminales. (2) *Calidad*: 100 % de rutas válidas en M; $\bar{\rho}$ de 1.025–1.049 y $\rho = 1$ exacto en 58–64 % de las consultas, pero el percentil 95 es 1.11–1.24: la meta $\rho \leq 1.1$ *aún no se cumple* en la cola (L2 en Q5). B1 empeora con k ($\bar{\rho} = 1$ con $k = 1$, 1.40–1.57 con $k = 6$) porque no optimiza el orden; B2 no conecta los terminales en 64 %, 86 % y 100 % de las consultas. (3) *Reparación*: las 150 consultas de M necesitaron reparaciones dirigidas (14–19 por consulta en promedio): reorientar el árbol no basta en una red con calles de un sentido, y por eso M tarda más que REF en el centro (290 frente a 119 ms). (4) *Contracción*: reduce los tokens de M 39 %, 65 % y 75 % según la zona; en B0 apenas 1–3 %, porque OSMnx ya simplifica el grafo. (5) *Formato*: JSON cuesta 2.05× los tokens del formato lista, que adoptamos.

Cuadro I
Experimento preliminar: medianas sobre 50 consultas por zona ($k = 1, 2, 3, 4, 6$; 10 c/u). Tokens en formato lista (`o200k_base`); tiempo y memoria pico del algoritmo, sin LLM; $\bar{\rho}$ y $\rho_{\text{máx}}$ sobre rutas válidas.

Zona (n)	Mét.	Tokens	ms	KB	Válidas	$\bar{\rho}$	$\rho_{\text{máx}}$
tec (591)	B0	10,247	5.9	361	—	—	—
	B1	508	1.6	122	50/50	1.250	1.942
	B2	4,055	6.9	191	18/50	1.008	1.033
	M	1,094	10.3	372	50/50	1.049	1.499
	REF	476	3.5	160	50/50	1.000	1.000
corredor (2,204)	B0	45,837	26.1	1,807	—	—	—
	B1	622	6.3	489	50/50	1.270	1.992
	B2	9,572	15.1	348	7/50	1.118	1.236
	M	1,352	30.5	1,659	50/50	1.025	1.234
	REF	537	13.2	631	50/50	1.000	1.000
centro (11,509)	B0	254,893	166.0	9,735	—	—	—
	B1	854	37.4	2,493	50/50	1.219	1.828
	B2	21,920	74.4	975	0/50	—	—
	M	2,136	290.0	8,185	50/50	1.047	1.481
	REF	702	119.3	4,858	50/50	1.000	1.000

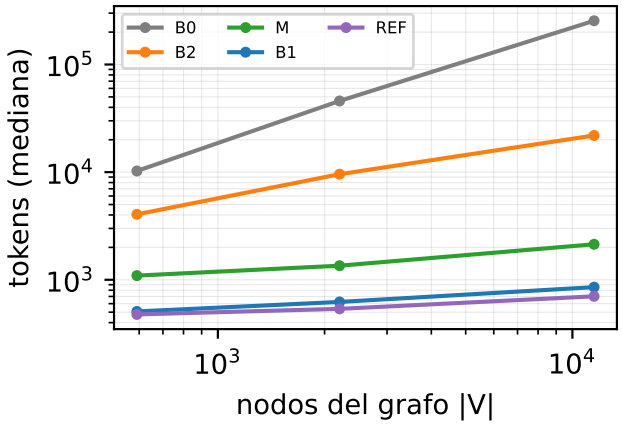


Figura 1. Tokens del prompt contra el tamaño del grafo (escala log–log).

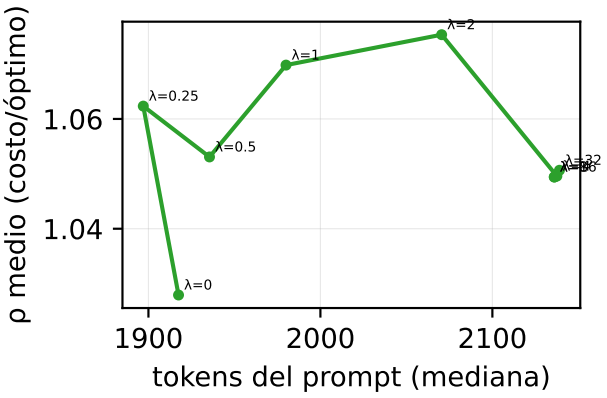


Figura 2. Barrido de λ en el corredor (20 consultas, $k = 4$): tokens (mediana) contra $\bar{\rho}$. Más λ no reduce tokens.

VIII. M2 — Evaluación determinista y robusta (diseñada; parte algorítmica implementada)

Protocolo. (1) *Consultas fijas*: generadas con semilla 2038 por zona, 3 zonas $\times$ 5 valores de $k\times$ 10 = 150 consultas, guardadas en el repositorio. (2) *Respuestas de referencia verificables*: R^* por Held–Karp sobre distancias exactas de Dijkstra en G. (3) *Métricas objetivas*: validez, ρ, cobertura de POI (algoritmo); coincidencia de secuencia de calles y alucinaciones (LLM). (4) *Repeticiones*: 5 ejecuciones del algoritmo por consulta para tiempo (mediana) y hash SHA-256 del texto; 10 llamadas al LLM por consulta. (5) *Control*: versión de modelo fijada, plantilla fija, $T \in \{0, 0.7\}$, `top_p`$= 1$, semilla, mismo tokenizador; grafos OSM cacheados en el repositorio para que los datos no cambien. (6) Baseline sin optimizar B0.

Determinismo contra variabilidad. La herramienta es determinista por diseño (empates rotos por id de nodo, iteración en orden ordenado, sin aleatoriedad). Lo verificamos: en las 603 salidas del experimento (150 consultas $\times$ 4 métodos, más B0 por zona), las 5 ejecuciones dieron el mismo hash SHA-256, y dos procesos con `PYTHONHASHSEED` distinto (1 y 999) produjeron los 603 hashes, tokens y ρ idénticos, igual que el barrido de λ. Por tanto, toda variación observada en la respuesta final se atribuye al LLM, que se medirá condicionando a una salida idéntica de la herramienta.

Experimento: barrido de λ (Fig. 2). Resultado negativo. Al subir λ de 0 a 32, la mediana de aristas del árbol baja de 100 a 81, pero los tokens *suben* de 1,918 a 2,139 y $\bar{\rho}$ empeora de 1.028 a 1.05–1.08: no hay frente de Pareto. Diagnóstico: $\tau(e)$ cobra cada arista por igual, pero tras la contracción una avenida larga cuesta una sola línea; penalizar el número de aristas lleva al árbol a calles con menos intersecciones pero más cambios de nombre, que es lo que realmente cuesta tokens. La corrección está en Q5 (L1).

IX. M3 — Preservación de calidad y asignación de recursos (planeado)

Objetivo: $\min f_1$ sujeto a umbrales de calidad. **Umbrales:** rutas válidas 100 %; cobertura de POI 100 %; $\rho \leq 1.10$ en el percentil 95; restricciones C respetadas 100 %; fidelidad del LLM (secuencia de calles exacta) $\geq 95\,\%$ a $T = 0$; consistencia ≥ 0.9 de acuerdo entre pares a $T = 0$. **Asignación:** B adaptativo según k y longitud de la ruta; selección de λ por relajación Lagrangiana; mejora L2 ($T \cup R^*$) para cumplir $\rho_{95} \leq 1.1$; proxy τ por cambios de calle (L1) para que λ sí controle f_1. Filtro de restricciones C como poda previa de aristas (peajes, tipo de vía de OSM).

X. M4 — Benchmark y sistema final (planeado)

Arquitectura final: servidor MCP con la herramienta `plan_route(origen, destino, pois, restricciones, presupuesto)`, un LLM con llamada a herramientas y el módulo de evaluación. **Clásicos comparados:** Dijkstra contra A* con heurística euclidiana; KMB [11] contra Mehlhorn y Dreyfus–Wagner exacto [16] ($\ell \leq 8$, para medir la brecha real de aproximación); Held–Karp contra vecino más cercano + 2-opt. **Dimensiones:** eficiencia (tokens, tiempo, memoria, costo en USD por consulta), calidad (validez, ρ, completitud), consistencia (acuerdo entre repeticiones), determinismo (hash de la salida, variabilidad del LLM) y escalabilidad (n hasta Monterrey–Saltillo, k hasta 12, lotes de consultas).

XI. Estado del repositorio y ruta de trabajo

Repositorio (`steiner-llm`): `src/` (grafo, algoritmos, serializador, métodos), `tests/` (6 pruebas), `experiments/` (experimento preliminar y figuras), `data/` (grafos cacheados), `results/` (CSV) y `paper/`. **Hecho:** M1 completo sin LLM; parte algorítmica de M2. **Pendiente:** integración MCP + LLM y medición de variabilidad (semanas 1–2), M3: umbrales, restricciones, mejoras L1 y L2 (semanas 3–4), M4: clásicos, escala Monterrey–Saltillo y benchmark final (semanas 5–6).

Referencias

[1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[2] Anthropic, "Model context protocol specification," https://modelcontextprotocol.io, 2024.

[3] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017.

[4] R. M. Karp, "Reducibility among combinatorial problems," in *Complexity of Computer Computations*. Plenum, 1972, pp. 85–103.

[5] M. Charikar, C. Chekuri, T.-y. Cheung, Z. Dai, A. Goel, S. Guha, and M. Li, "Approximation algorithms for directed Steiner problems," *Journal of Algorithms*, vol. 33, no. 1, pp. 73–91, 1999.

[6] OpenAI, "tiktoken: a fast BPE tokeniser," https://github.com/openai/tiktoken, 2023.

[7] K. Mehlhorn, "A faster approximation algorithm for the Steiner problem in graphs," *Information Processing Letters*, vol. 27, no. 3, pp. 125–128, 1988.

[8] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[9] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," *Journal of the Society for Industrial and Applied Mathematics*, vol. 10, no. 1, pp. 196–210, 1962.

[10] M. L. Fredman and R. E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms," *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.

[11] L. Kou, G. Markowsky, and L. Berman, "A fast algorithm for Steiner trees," *Acta Informatica*, vol. 15, pp. 141–145, 1981.
[12] J. Byrka, F. Grandoni, T. Rothvoß, and L. Sanità, "Steiner tree approximation via iterative randomized rounding," *Journal of the ACM*, vol. 60, no. 1, pp. 6:1–6:33, 2013.
[13] P. Lewis, E. Perez, A. Piktus *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
[14] P. Klein and R. Ravi, "A nearly best-possible approximation algorithm for node-weighted Steiner trees," *Journal of Algorithms*, vol. 19, no. 1, pp. 104–115, 1995.
[15] N. Garg, G. Konjevod, and R. Ravi, "A polylogarithmic approximation algorithm for the group Steiner tree problem," *Journal of Algorithms*, vol. 37, no. 1, pp. 66–84, 2000.
[16] S. E. Dreyfus and R. A. Wagner, "The Steiner problem in graphs," *Networks*, vol. 1, no. 3, pp. 195–207, 1971.

APÉNDICE

Python 3.13; versiones fijadas en `requirements.txt`; grafos OSM versionados en `data/`.

```
python -m venv .venv
pip install -r requirements.txt
python -m pytest
python experiments/run_prelim.py
python experiments/plots.py
```

Para el determinismo entre procesos se corre dos veces con `PYTHONHASHSEED=1` y `=999` y se comparan los hashes de la columna `sha` de `results/prelim.csv`.